

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ
Кафедра програмної інженерії

Звіт

З лабораторної роботи 3
«Модульне тестування програмного коду (Unit Testing)»

Виконав: студент групи ПЗПІ-25-4
Гоков Дмитро Олегович
Перевірив: В. О. Гребенюк

Харків 2026

3.1 Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

3.2 Посилання

Посилання на Git-репозиторій з кодом модуля та тестами:

<https://github.com/mk1234717/MenuQR/tree/gokov-work/lr3>

3.3 Код модуля Для написання коду був використаний Gemini, промпт:

You are a senior software engineer. I need to implement a Python module for unit testing lab work based on the following UML class diagram (from a restaurant QR menu system).

The main class is Order with a helper class OrderItem and an OrderStatus (Enum). Order has attributes: id: UUID, table_number: int, items: List[OrderItem], status: OrderStatus, discount_percent: Decimal. OrderItem has attributes: name: String, price: float, quantity: int.

Add additional fields if needed for business logic.

Requirements:

- Implement at least 3 non-trivial methods (getters/setters don't count). Each method must contain:
 - o conditional logic (if/else, switch) or loops
 - o exception handling (raise ValueError, RuntimeError, etc.)
 - o meaningful business rules (adding items with item quantity limits, applying restaurant promo codes, total calculation with tips, etc.)
- Use Python with proper type hints.

- Use Decimal for price and total calculations (avoid float precision issues). Provide full class code with docstrings.
- Do NOT include any file I/O, database, or external API calls - pure logic for unit testing.
- The class must be easy to test with pytest. Show me the final code.

Код:

```
import uuid
from typing import List
from decimal import Decimal, ROUND_HALF_UP
from enum import Enum

class OrderStatus(Enum):
    PENDING = "pending"
    COOKING = "cooking"
    READY = "ready"
    COMPLETED = "completed"

class OrderItem:
    """Елемент страви у замовленні."""
    def __init__(self, name: str, price: float, quantity: int):
        if not name or not name.strip():
            raise ValueError("Назва страви не може бути порожньою.")
        if price <= 0:
            raise ValueError("Ціна страви повинна бути більшою за 0.")
        if quantity <= 0 or quantity > 20:
            raise ValueError("Кількість однієї страви повинна бути в межах від 1 до 20.")

        self.name = name.strip()
        self.price = Decimal(str(price))
        self.quantity = quantity

class Order:
    """Клас замовлення цифрового QR-меню."""
    def __init__(self, table_number: int):
        if table_number <= 0 or table_number > 50:
            raise ValueError("Номер столика повинен бути в межах від 1 до 50.")

        self.id = uuid.uuid4()
        self.table_number = table_number
        self.items: List[OrderItem] = []
        self.status: OrderStatus = OrderStatus.PENDING
        self.discount_percent = Decimal('0')
```

```

# Метод №1: Додавання страви до замовлення (Логіка циклів, умов та
винятків)
def add_item(self, name: str, price: float, quantity: int) -> None:
    if self.status != OrderStatus.PENDING:
        raise RuntimeError("Не можна редагувати замовлення, яке вже
готується або виконано.")

    for item in self.items:
        if item.name == name.strip():
            new_qty = item.quantity + quantity
            if new_qty > 20:
                raise ValueError("Сумарна кількість однієї страви не може
перевищувати 20.")
            item.quantity = new_qty
            return

    self.items.append(OrderItem(name, price, quantity))

# Метод №2: Застосування промокоду (Валідація та умовна логіка)
def apply_promo_code(self, promo_code: str) -> None:
    if self.status != OrderStatus.PENDING:
        raise RuntimeError("Промокод можна застосувати тільки до нового
замовлення.")

    valid_promos = {
        "SAVE10": Decimal('10'),
        "EATLOCAL": Decimal('15'),
        "VIP25": Decimal('25')
    }

    cleaned_code = promo_code.strip().upper()
    if cleaned_code not in valid_promos:
        raise ValueError(f"Недійсний або неіснуючий промокод:
{promo_code}")

    self.discount_percent = valid_promos[cleaned_code]

# Метод №3: Розрахунок фінальної вартості з чайовими та знижкою
def calculate_total(self, tip_percent: float = 0.0) -> float:
    if tip_percent < 0 or tip_percent > 50:
        raise ValueError("Чайові повинні бути в діапазоні від 0% до 50%.")

    if not self.items:
        return 0.0

    subtotal = sum(item.price * item.quantity for item in self.items)

```

```

if self.discount_percent > 0:
    subtotal -= subtotal * (self.discount_percent / Decimal('100'))

tips = subtotal * (Decimal(str(tip_percent)) / Decimal('100'))
total = subtotal + tips

return float(total.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP))

```

3.4 Таблиця проєктування тестів

№	Метод / Компонент	Опис тест-кейсу	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
1	add_item	Успішне додавання нової унікальної страви	name="Піца", price=180.0, quantity=2	Елемент додано, len(items) == 1, назва та кількість збігаються	EP (Позитивний)	pass
2	add_item	Редагування замовлення у невідповідному статусі	status = OrderStatus.COOKING, додавання страви	Виклик винятку RuntimeError ("Не можна редагувати замовлення..")	EP (Негативний)	pass
3	add_item	Перевірка нижньої межі кількості страв	quantity = 1 (для нової страви "Сік")	Страва успішно додана, кількість рівна 1	BVA (Нижня межа)	pass
4	add_item	Перевірка верхньої межі кількості страв	quantity = 20 (для страви "Кава")	Страва успішно додана, кількість рівна 20	BVA (Верхня межа)	pass
5	add_item	Перевищення	До страви з qty=20	Виклик винятку	BVA (За верхньою межею)	pass

№	Метод / Компонент	Опис тест-кейсу	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
		ліміту кількості страв через накопичення	додати ще quantity = 1	ValueError ("сумарна кількість... не може перевищувати 20")	ю межею)	
6	apply_promo_code	Застосування валідного промокоду закладу	promo_code = "EATLOCAL"	Знижка успішно встановлена, discount_percent == 15	EP (Позитивний)	pass
7	apply_promo_code	Спроба використання неіснуючого промокоду	promo_code = "INVALID_CODE"	Виклик винятку ValueError ("Недійсний або неіснуючий промокод...")	EP (Негативний)	pass
8	calculate_total	Розрахунок фінальної вартості для порожнього замовлення	items = []	Повертає значення 0.0	EP (Порожній стан)	pass
9	calculate_total	Комплексний розрахунок вартості зі знижкою та чайовим	Сума страв 200.0, промокод SAVE10 (-10%), tip_percent = 10.0	Повертає точне значення 198.0 (з урахуванням округлення)	EP (Комбінований)	pass

№	Метод / Компонент	Опис тест-кейсу	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
		и				
10	calculate_total	Розрахунок з чайовим и на нижній межі	tip_percent = 0.0 (для замовлення на 500.0)	Повертає чисту вартість 500.0 без чайових	BVA (Нижня межа)	pass
11	calculate_total	Розрахунок з чайовим и на верхній межі	tip_percent = 50.0 (для замовлення на 500.0)	Повертає вартість 750.0 (+50% чайових)	BVA (Верхня межа)	pass
12	calculate_total	Перевірка відсотків в чайових за нижньою межею	tip_percent = -0.01	Виклик винятку ValueError ("Чайові повинні бути в діапазоні...")	BVA (За межею)	pass
13	calculate_total	Перевірка відсотків в чайових за верхньою межею	tip_percent = 50.01	Виклик винятку ValueError ("Чайові повинні бути в діапазоні...")	BVA (За межею)	pass
14	OrderItem	Створення страви з некоректним (порожнім) ім'ям	name = " ", price = 10.0, quantity = 1	Виклик винятку ValueError ("Назва страви не може бути порожньою.")	EP (Негативний)	pass

№	Метод / Компонент	Опис тест-кейсу	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
15	Order	Перевірка номера столика за нижньою межею	table_number = 0	Виклик винятку ValueError ("Номер столика повинен бути в межах...")	BVA (За нижньою межею)	pass
16	Order	Перевірка номера столика за верхньою межею	table_number = 51	Виклик винятку ValueError ("Номер столика повинен бути в межах...")	BVA (За верхньою межею)	pass

3.5 Код модульних тестів:

```
import pytest
from pytest import approx
from order import Order, OrderItem, OrderStatus

# --- Тести методу add_item ---

def test_add_item_success():
    """EP: Успішне додавання нової страви у замовлення."""
    # Arrange
    order = Order(table_number=5)

    # Act
    order.add_item(name="Піца", price=180.0, quantity=2)

    # Assert
    assert len(order.items) == 1
    assert order.items[0].name == "Піца"
    assert order.items[0].quantity == 2

def test_add_item_wrong_status():
```

```

    """EP: Спроба зміни замовлення у невідповідному статусі ->
    RuntimeError."""
    # Arrange
    order = Order(table_number=5)
    order.status = OrderStatus.COOKING

    # Act & Assert
    with pytest.raises(RuntimeError, match="Не можна редагувати замовлення"):
        order.add_item(name="Піца", price=180.0, quantity=1)

def test_add_item_quantity_boundaries():
    """BVA: Перевірка меж кількості страв (1, 20, 21)."""
    order = Order(table_number=1)

    # Межа 1 - успішно
    order.add_item("Сік", 40.0, 1)
    assert order.items[0].quantity == 1

    # Межа 20 - успішно для нового елементу
    order.add_item("Кава", 50.0, 20)
    assert order.items[1].quantity == 20

    # Межа 21 (або накопичення > 20) -> ValueError
    with pytest.raises(ValueError, match="кількість однієї страви не може
    перевищувати 20"):
        order.add_item("Кава", 50.0, 1)

# --- Тести методу apply_promo_code ---

def test_apply_promo_code_valid():
    """EP: Валідний промокод встановлює правильну знижку."""
    # Arrange
    order = Order(table_number=12)

    # Act
    order.apply_promo_code("EATLOCAL")

    # Assert
    assert order.discount_percent == 15

def test_apply_promo_code_invalid():
    """EP: Невалідний промокод викликає виняток ValueError."""
    # Arrange
    order = Order(table_number=12)

    # Act & Assert
    with pytest.raises(ValueError, match="Недійсний або неіснуючий промокод"):
        order.apply_promo_code("INVALID_CODE")

```

```
# --- Тести методу calculate_total ---

def test_calculate_total_empty():
    """EP: Розрахунок суми для порожнього замовлення рівний 0."""
    # Arrange
    order = Order(table_number=3)

    # Act
    total = order.calculate_total()

    # Assert
    assert total == approx(0.0)

def test_calculate_total_with_discount_and_tips():
    """EP: Комплексний розрахунок вартості зі знижкою та чайовими."""
    # Arrange
    order = Order(table_number=3)
    order.add_item("Борщ", 100.0, 2) # 200.0
    order.apply_promo_code("SAVE10") # -10% -> 180.0

    # Act
    total = order.calculate_total(tip_percent=10.0)

    # Assert
    assert total == approx(198.0)

def test_calculate_total_tip_boundaries():
    """BVA: Перевірка меж відсотків чайових (0%, 50%, -0.01%, 50.01%)."""
    order = Order(table_number=2)
    order.add_item("Стейк", 500.0, 1)

    # Нижня межа 0%
    assert order.calculate_total(tip_percent=0.0) == approx(500.0)

    # Верхня межа 50%
    assert order.calculate_total(tip_percent=50.0) == approx(750.0)

    # Поза нижньою межею
    with pytest.raises(ValueError, match="Чайові повинні бути в діапазоні"):
        order.calculate_total(tip_percent=-0.01)

    # Поза верхньою межею
    with pytest.raises(ValueError, match="Чайові повинні бути в діапазоні"):
        order.calculate_total(tip_percent=50.01)

def test_order_item_invalid_name():
    """EP/Негативний: Створення елемента замовлення без назви."""
```

```

    with pytest.raises(ValueError, match="Назва страви не може бути
порожньою"):
        OrderItem(name="   ", price=10.0, quantity=1)

def test_order_invalid_table():
    """BVA/Негативний: Некоректний номер столика (0 або 51)."""
    with pytest.raises(ValueError, match="Номер столика повинен бути в
межах"):
        Order(table_number=0)

```

Код для запуску тесту:

```

Name      Stmts  Miss  Cover   Missing
-----
order.py    59      5    92%    18, 20, 48-49, 56
-----
TOTAL       59      5    92%
===== 10 passed in 0.20s =====
PS C:\Users\Dima\Desktop\OPR\Задачи>

```